

Bancos de Dados Não-Relacionais

Aula 03 (Sábado Letivo): Prática de MongoDB & Redis (CLI, GUI e Python)

A presentation by

Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

Roteiro de Atividades – Sábado Letivo (Não-Presencial)

Ambiente Remoto e Arquitetura Híbrida

MongoDB Direto no Banco: CLI & GUI

Redis Direto no Banco: CLI & Redis Insight

Integração Complementar com Python: Cache-Aside

Roteiro da Lista de Exercícios Práticos

Ambiente Remoto e Arquitetura Híbrida

Contexto Pedagógico: Sábado Letivo Autônomo

Objetivo Central da Prática

Dominar CRUD no MongoDB e no Redis, unindo-os através do padrão Cache-Aside.

- **Nivelamento Prático:** Domínio direto via CLI e GUIs.
- **Arquitetura:** Persistência durável vs RAM de baixa latência.

Duas Trilhas de Execução

1. Nuvem + Desktop GUI:
MongoDB Atlas Free + Compass e Redis Cloud + Redis Insight.

2. Containers Locais (Docker):
Provisionamento com comando único:

```
docker compose up -d
```

Opção 1: Setup em Nuvem com GUIs Oficiais

MongoDB Atlas & Compass

- 1 Criar conta gratuita no **MongoDB Atlas**.
- 2 Criar cluster compartilhado **M0** (512 MB).
- 3 Em Database Access, criar usuário/senha.
- 4 Em Network Access, liberar seu IP (0.0.0.0/0).
- 5 Baixar o **MongoDB Compass** e conectar via string SRV (`mongodb+srv://...`).

Redis Cloud & Redis Insight

- 1 Baixar o **Redis Insight** (redis.io/insight).
- 2 Conectar em instância local (`localhost:6379`) ou Redis Cloud gratuito (30 MB).
- 3 Acessar navegador de chaves em árvore (Key Tree) e terminal CLI integrado.
- 4 Analisar estruturas e consumo de memória RAM.

Opção 2: Setup Local via Docker Compose

Comando Único de Inicialização

```
docker compose up -d
```

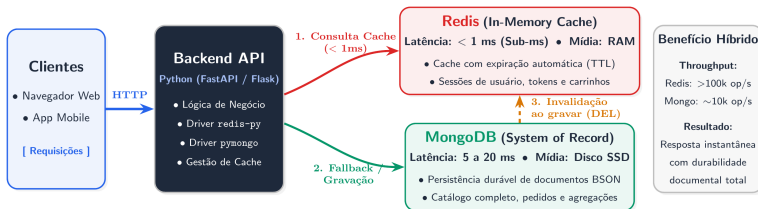
Serviços Subindo no Host:

- **MongoDB 8.0** (:27017) + **Express** (:8081)
- **Redis 7.4** (:6379) + **Insight** (:5540)

Vantagens do Ambiente Local

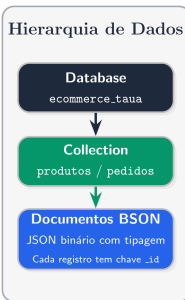
- Reprodutível em Linux, Windows e macOS.
- Volumes persistentes pré-configurados.
- Não polui o sistema operacional host.
- Para encerrar: `docker compose down`.

Visão Arquitetural Híbrida: MongoDB + Redis



MongoDB Direto no Banco: CLI & GUI

Modelo Documental BSON no MongoDB Compass



Anatomia BSON no Compass

```
{
  _id: ObjectId("6709a1..."),
  sku: "NOT-DEL-01",
  nome: "Notebook Dell 15",
  preco: 4299.90,
  estoque: 14,
  especificacoes: {
    cpu: "Core i7",
    ram_gb: 16,
    ssd_gb: 512
  },
  tags: ["trabalho", "dev"],
  ativo: true
}
```

Operações CRUD no Banco

[C] Create

insertOne() ou botão "Add Data"

[R] Read

find({ preco: { \$lt: 5000 } }) na Filter

[U] Update

updateOne() ou ícone de lápis "Edit"

[D] Delete

deleteOne() ou ícone de lixeira "Remove"

Operando no MongoDB Compass (Interface Gráfica)

Passo a Passo Visual no Compass

- 1 Conectar em `localhost:27017` (ou Atlas).
- 2 Criar banco `ecommerce_taua / produtos`.
- 3 Clicar em **Add Data** → **Insert Document**.
- 4 Alternar para visualização **JSON** e colar.
- 5 Clicar em **Insert** para persistir.

Consultas na Barra Filter

- **Filtro de Categoria:**
`{ categoria: "Perifericos" }`
- **Filtro de Preço:**
`{ preco: { $lt: 1000 } }`
- **Projeção (Project):**
`{ nome: 1, preco: 1, _id: 0 }`

Terminal mongosh: Criação e Inserção Individual

Seleção de Banco e Inserção com insertOne:

```
// 1. Criar e selecionar o banco de dados
use ecommerce_taua

// 2. Inserir documento individual rico com atributos aninhados
db.produtos.insertOne({
  sku: "NOT-DEL-01",
  nome: "Notebook Dell Inspiron 15",
  categoria: "Informatica",
  preco: 4299.90,
  estoque: 14,
  especificacoes: { processador: "Core i7", ram_gb: 16, ssd_gb:
                    512 },
  tags: ["trabalho", "desenvolvimento", "bivolt"],
  ativo: true
})
```

Terminal mongosh: Inserção em Lote (insertMany)

Inserção Múltipla com Eficiência de Rede (Bulk Insert):

```
db.produtos.insertMany([
  { sku: "MOU-LOG-02", nome: "Mouse Logitech MX",
    categoria: "Perifericos", preco: 589.00, estoque: 30 },
  { sku: "TEC-MEC-03", nome: "Teclado Keychron K2",
    categoria: "Perifericos", preco: 699.50, estoque: 20 },
  { sku: "MON-DEL-04", nome: "Monitor Dell 27 4K",
    categoria: "Monitores", preco: 2890.00, estoque: 8 }
])
```

- Reduz round-trips de rede agrupando registros em pacote único.
- Retorna um array com os `insertedIds` gerados automaticamente.

Terminal mongosh: Consultas Básicas e Projeção

Filtros de Comparação e Seleção de Campos:

```
// 1. Buscar todos os documentos projetando apenas sku, nome e
    preco
db.produtos.find(
  {},
  { _id: 0, sku: 1, nome: 1, preco: 1 }
)

// 2. Filtrar produtos por categoria e preco menor que R$ 650 (
    $lt)
db.produtos.find({
  categoria: "Perifericos",
  preco: { $lt: 650.00 }
})
```

Terminal mongosh: Consultas em Arrays e Subdocumentos

Busca em Estruturas Aninhadas (Dot-Notation):

```
// 1. Busca exata dentro de array de tags
db.produtos.find({ tags: "desenvolvimento" })

// 2. Consulta em objeto aninhado usando notacao de ponto
db.produtos.find({ "especificacoes.ram_gb": { $gte: 16 } })

// 3. Ordenacao decrescente por preco e limite
db.produtos.find().sort({ preco: -1 }).limit(3)
```

Terminal mongosh: Atualizações Atômicas e Remoção

Modificações (\$set, \$inc, \$push)

```
// Atualizar preco e adicionar tag
db.produtos.updateOne(
  { sku: "NOT-DEL-01" },
  {
    $set: { preco: 3899.00 },
    $push: { tags: "liquidacao" }
  }
)

// Incrementar estoque de forma
  atomica
db.produtos.updateOne(
  { sku: "MOU-LOG-02" },
  { $inc: { estoque: 5 } }
)
```

Deleção e Índices

```
// Remover documento individual
db.produtos.deleteOne({
  sku: "CAB-USBC-05"
})

// Remover produtos inativos
db.produtos.deleteMany({
  ativo: false
})

// Criar indice para busca veloz
db.produtos.createIndex({
  categoria: 1
})
```

Redis Direto no Banco: CLI & Redis Insight

As 5 Estruturas de Dados Fundamentais do Redis

1. Strings

Texto, Número, JSON

```
"session:token"  
→ "jwt_987xyz"  
  
"views:prod"  
→ 126
```

Comandos Chave:

SET, GET, INCR
EXPIRE, SETNX

Uso Prático:

Sessões de Usuário,
Contadores e Cache

2. Hashes

Objeto Campo-Valor

```
"cart:user_42"  
→ NOT-DEL: 1  
→ MOU-LOG: 2  
  
"user:101" → nome
```

Comandos Chave:

HSET, HGETALL
HINCRBY, HDEL

Uso Prático:

Carrinho de Compras,
Perfil de Usuário

3. Lists

Fila Encadeada

```
"fila:pedidos"  
→ [PED-1, PED-2]  
  
Entra pela cauda,  
sai pela cabeça
```

Comandos Chave:

LPUSH, RPUSH
LPOP, LRANGE

Uso Prático:

Filas de Tarefas,
Histórico Recente

4. Sets

Conjuntos Únicos

```
"tags:hardware"  
→ {"ram", "ssd",  
"cooler"}  
  
Sem duplicatas!
```

Comandos Chave:

SADD, SMEMBERS
SISMEMBER, SREM

Uso Prático:

Tags de Produtos,
Usuários Conectados

5. Sorted Sets

Conjunto com Score

```
"ranking:vendas"  
→ 70: Cabo USB  
→ 50: Mouse MX  
→ 12: Dell 15
```

Comandos Chave:

ZADD, ZRANGE
ZREVRANGE, ZINCRBY

Uso Prático:

Leaderboards,
Mais Vendidos

Operando com Redis Insight (Interface Gráfica)

Recursos Visuais do Redis Insight

- 1 **Key Tree:** organiza chaves hierárquicas separadas por :.
- 2 **Editor Visual:** inspeciona e edita Hashes, Listas e Sets em tempo real.
- 3 **Memory Profiler:** analisa consumo de memória RAM por padrão de chave.
- 4 **CLI Embutido:** terminal interativo acessível no rodapé.

Padrão de Nomes de Chaves

Convenções recomendadas:

- entidade:id
Ex: user:101
- entidade:id:subtipo
Ex: cart:user:42
- servico:escopo:chave
Ex: cache:prod:01

Terminal redis-cli: Strings, TTL e Contadores

Operações Chave-Valor Básicas e Expiração Automática:

```
// 1. Sessao temporaria com TTL de 120s
SET session:ads_user_42 "token_jwt_987" EX 120
GET session:ads_user_42           // Retorna o valor
TTL session:ads_user_42           // Retorna segundos restantes

// 2. Contador atomico de acessos
INCR views:produto:NOT-DEL-01
INCRBY views:produto:NOT-DEL-01 25

// 3. Trava distribuida atomica (Lock idempotente)
SET lock:pedido:9001 "worker_1" NX EX 30
```

Terminal redis-cli: Hashes e Carrinho de Compras

Modelagem de Objetos e Carrinhos com Hashes:

```
// 1. Criar entidade com multiplos campos
HSET user:101 nome "Reginaldo Fernandes" campus "Taua" nivel "
    Docente"

// 2. Consultar campo individual e o hash completo
HGET user:101 nome
HGETALL user:101

// 3. Modelagem de Carrinho de Compras (SKU -> Quantidade)
HSET cart:session_taua_77 "NOT-DEL-01" 1 "MOU-LOG-02" 2

// 4. Incrementar item de forma atomica
HINCRBY cart:session_taua_77 "MOU-LOG-02" 1
```

Terminal redis-cli: Listas, Conjuntos e Rankings

Filas FIFO com Lists

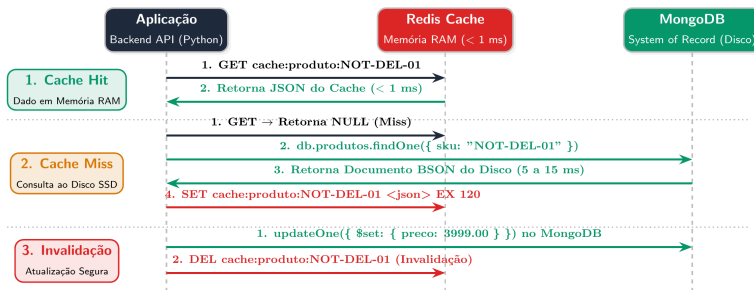
```
// Produtor enfileira na cauda  
RPUSH fila:pedidos "PED-1" "PED-2"  
  
// Consumidor retira da cabeça  
LPOP fila:pedidos  
  
// Consultar fila sem remover  
LRANGE fila:pedidos 0 -1
```

Rankings com Sorted Sets

```
// Adicionar score de vendas  
ZADD ranking 45 "MOU-LOG-02"  
ZADD ranking 12 "NOT-DEL-01"  
ZADD ranking 70 "CAB-USBC-05"  
  
// Top 3 mais vendidos (decrecente)  
ZREVRANGE ranking 0 2 WITHSCORES
```

Integração Complementar com Python: Cache-Aside

O Padrão Arquitetural Cache-Aside (Lazy Loading)



Cache-Aside em Python: Verificação e Cache Hit

Fluxo de Consulta em Memória RAM (Sub-milissegundo):

```
def obter_produto(sku: str, mongo_col, redis_cli, ttl=120):  
    cache_key = f"cache:produto:{sku}"  
  
    # 1. Tenta recuperar da memoria RAM (Redis)  
    dado_cache = redis_cli.get(cache_key)  
    if dado_cache:  
        return json.loads(dado_cache), "CACHE_HIT"
```

- **Cache Hit:** Resposta imediata da RAM (< 1 ms) sem sobrecarregar o MongoDB.

Cache-Aside em Python: Cache Miss e População

Fallback no MongoDB e Armazenamento com TTL:

```
# 2. CACHE MISS: Busca na fonte da verdade (MongoDB)
produto_db = mongo_col.find_one({"sku": sku}, {"_id": 0})
if not produto_db:
    return None, "NOT_FOUND"

# 3. Popula o cache no Redis com tempo de vida definido (
    TTL)
redis_cli.set(cache_key, json.dumps(produto_db), ex=ttl)
return produto_db, "CACHE_MISS"
```

- **Lazy Loading Idempotente:** O cache só é abastecido sob demanda, otimizando a memória RAM.

Invalidação de Cache e Prevenção de Dados Obsoletos

Atualização com Invalidação

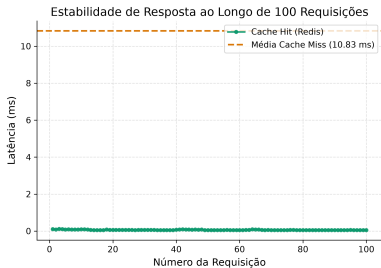
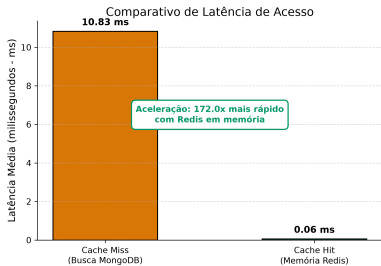
```
def atualizar_preco(sku, preco, col, r):  
    # 1. Atualiza MongoDB  
    col.update_one(  
        {"sku": sku},  
        {"$set": {"preco": preco}}  
    )  
    # 2. Invalida no Redis  
    r.delete(f"cache:prod:{sku}")
```

O Risco de Stale Read Sem invalidação ativa imediata:

- Usuários visualizam dados defasados até o TTL expirar.
- O comando `DEL` força o próximo acesso a buscar o valor atualizado na fonte oficial.

Resultados Empíricos: Benchmarking de Latência

IFCE Campus Tauá • ADS16 - Banco de Dados Não-Relacionais (Aula 03)



Roteiro da Lista de Exercícios Práticos

Estrutura da Lista de Exercícios (Entrega Sábado Letivo)

Bloco 1: MongoDB (Sem Python)

- **Ex 1 (Modelagem):** Coleção com 5 documentos ricos no `mongosh`.
- **Ex 2 (Consultas):** 3 buscas com operadores `$gt`, `$in` e dot-notation.
- **Ex 3 (GUI Compass):** Criar índice e analisar na aba *Explain Plan*.

Bloco 2: Redis (Sem Python)

- **Ex 4 (Sessão & TTL):** 3 tokens com TTL de 90s no `redis-cli`.
- **Ex 5 (Hashes - Carrinho):** Carrinho com `HSET` e `HINCRBY`.
- **Ex 6 (Redis Insight):** Print da árvore de chaves (Key Tree) e análise de RAM.

Bloco 3: Integração e Diretrizes de Submissão

Bloco 3: Integração com Python

- **Ex 7 (Cache-Aside):** Consulta no MongoDB com cache no Redis (TTL de 45s).
- **Ex 8 (Invalidação):** Atualização com remoção imediata (DEL) do Redis.

Critério de Avaliação

Autonomia, prints das GUIs e código documentado.

Instruções de Entrega

- **Prazo:** Até a sexta-feira anterior à Aula 04.
- **Formato:** PDF com comandos e prints ou repositório GitHub.
- **Envio:** SIGAA / Classroom.

Dúvidas & Suporte Remoto

IFCE Campus Tauá – ADS16 (Sábado Letivo)

Atendimento via Fórum SIGAA / GitHub Classroom

reginaldo.fernandes@ifce.edu.br